



Assiut University
Faculty of Computers & Information
Computer Science Department



Intelligent Systems

DR.SARA SALAH MOHAMED

Grading

Assessment	Grade %
Final Exam	50%
Class Work	15%
Mid-Term Exam	15%
Project	20%

Text Book

INTELLIGENT SYSTEMS REFERENCE LIBRARY
Volume 17

Crina Grosan
Ajith Abraham

Intelligent Systems

A Modern Approach

 Springer

Content

◀ Evolutionary Algorithms

Introduction

▶ How to Build an Evolutionary Algorithm?

◀ Genetic Algorithms

Representing the
Individuals

Selection Mechanisms

Selection Mechanisms

Variation Operators

Population Models

Survivor Selection and
Reinsertion

The Basic Genetic
Algorithm

Evolutionary Algorithms

- In nature, evolution is mostly determined by natural selection of different individuals competing for resources in the environment. Those individuals that are better are more likely to survive and propagate their genetic material.
- The encoding for genetic information (genome) is done in a way that admits reproduction, which results in offspring that are genetically identical to the parent. reproduction allows some exchange and re-ordering of chromosomes, producing offspring that contain a combination of information from each parent. This is the recombination operation, which is often referred to as crossover because of the way strands of chromosomes cross over during the exchange.
- The diversity in the population is achieved by mutation operation.

Usually grouped under the term evolutionary computation or evolutionary algorithms:

- genetic algorithms
- evolution strategies
- evolutionary programming
- genetic programming .
- **They all share a common conceptual base of simulating the evolution of individual structures via processes of selection, recombination and mutation reproduction and thereby producing better solutions.**

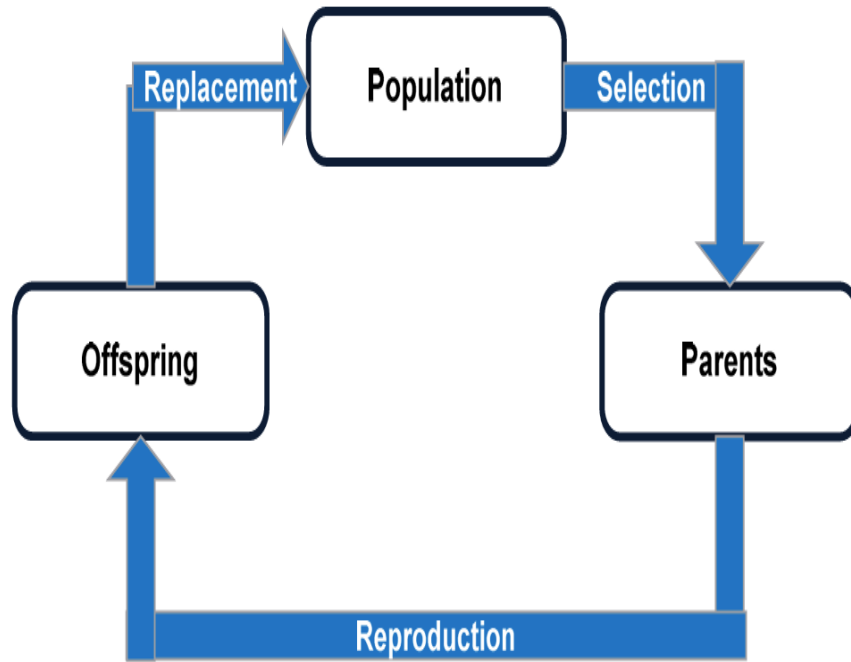
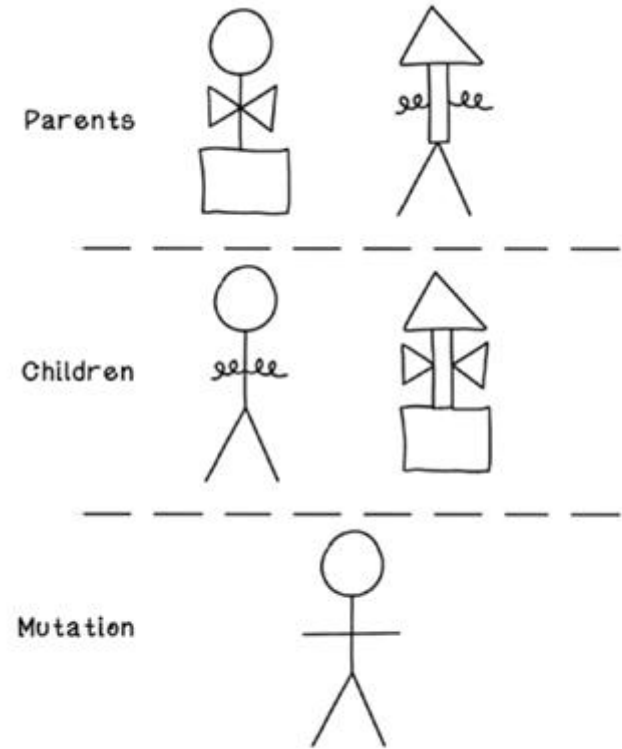


Fig. 14.1 Evolutionary scheme.



A simple example of reproduction and mutation

The correspondence between natural evolution and problem solving

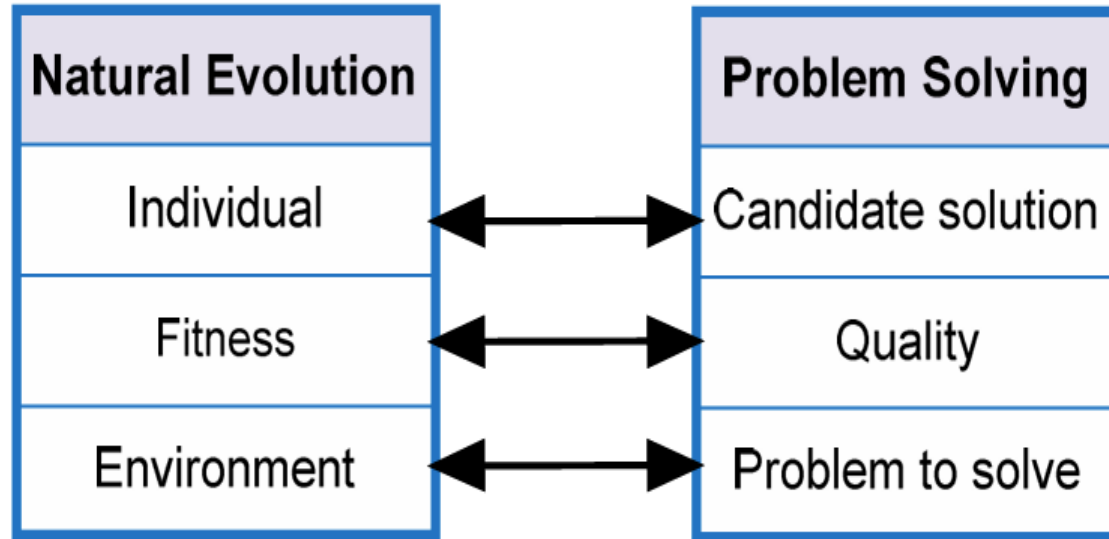


Fig. 14.2 Correspondence between natural evolution principles and problem solving.

Evolutionary Algorithm

Step 1. Set $t = 0$;

Step 2. Randomly initialize the population $P(t)$.

Repeat

 Step 3.1. Evaluate individuals from $P(t)$;

 Step 3.2. Selection on $P(t)$

 Let $P'(t)$ be the set of selected individuals.

 Step 3.3. Crossover on $P'(t)$;

 Step 3.4. Mutation on $P'(t)$.

 Step 3.5. Survival on $P'(t)$.

 Step 3.6. $t = t + 1$.

Until $t = \text{number of iterations}$

The variants of evolutionary algorithms differ only in technical details

- in the case of Genetic Algorithms (GAs) the solutions are represented as strings over a finite alphabet;
- solutions are represented as real-valued vectors in Evolutionary (Evolution) Strategies (ES);
- in the case of Evolutionary Programming (EP), solutions are finite state machines;
- Genetic Programming (GP) uses trees for solution representation.

How to Build an Evolutionary Algorithm?

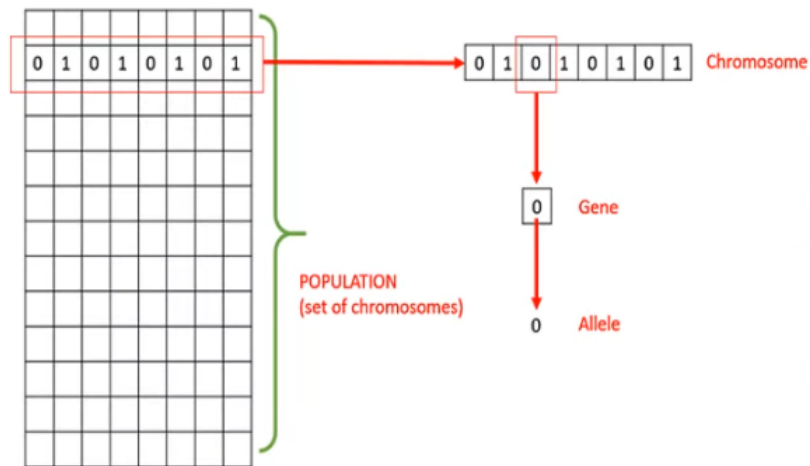
In order to build an evolutionary algorithm there are a number of steps that we have to perform:

- design a representation;
- find a way to initialize the population;
- design a way of mapping a genotype to a phenotype;
- design a way of evaluating an individual;
- decide how to select individuals to be parents (design a selection mechanism);
- design suitable mutation operators;
- design suitable recombination operators;
- decide how to select individuals to be replaced;
- decide when to stop the algorithm.

Genetic Algorithms

- The most widely known type of evolutionary algorithms and probable the most used are genetic algorithms.
- Simple to implement and use
- one of the most suitable techniques for any kind of optimization problem

Basic Terminology of GA



- **Population** – subset of all the possible solutions to the given problem.
- **Chromosomes** – one such solution to given problem
- **Gene** – one element position of a chromosome.
- **Allele** – value a gene takes for particular chromosome
- **Genotype** – population in the computation space.
- **Phenotype** – population in the actual real world solution space.
- **Decoding** – transforming a solution from the genotype to the phenotype space
- **Encoding** – transforming from the phenotype to genotype space.

Designing a Representation

- Representation here refers to a way of representing an individual as a genotype. An individual is also known as chromosome and it is composed of a set of genes.
- Representation of individuals is one of the most important steps in designing a genetic algorithm.
- For one problem there can be multiple possible representations; thus, we have to be able to decide which one is more adequate to satisfy our needs and requirements.
- The most important types of representing an individual:
 - binary representation;
 - integer representation;
 - real valued or floating- point representation;
 - order based or permutation representation.

1-Binary Representation

Binary representation is the most classical and fundamental way to encode solutions in genetic algorithms. Each solution (individual) is represented as a string of bits (0s and 1s).

```
Chromosome = [1, 0, 1, 1, 0, 0, 1, 0]  
              ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  
              Genes (each is a single bit)
```

Why Binary Representation is Popular

- Simplicity - Easy to implement and understand
- Natural for crossover - Bits can be easily combined
- Hardware analogy - Mimics digital computer architecture

1-Binary Representation

The Knapsack Problem for a Backpacking Trip

[1, 0, 1, 1, 0, 1, 0, 0] means:

- Tent: Yes (1)
- Sleeping Bag: No (0)
- Food: Yes (1)
- Stove: Yes (1)
- Water Filter: No (0)
- First Aid Kit: Yes (1)
- Camping Chair: No (0)
- Extra Clothes: No (0)

Total weight = 4 + 5 + 2 + 2 = 13 kg (within limit ✓)

Total value = 9 + 7 + 5 + 4 = 25

ICON	ITEM	WEIGHT	VALUE
	Tent	4kg	9
	Sleeping Bag	3kg	8
	Food	5kg	7
	Stove	2kg	5
	Water Filter	1kg	6
	First Aid	2kg	4
	Camping Chair	3kg	3
	Extra Clothes	2kg	4

Crossover or Recombination for Binary Representation

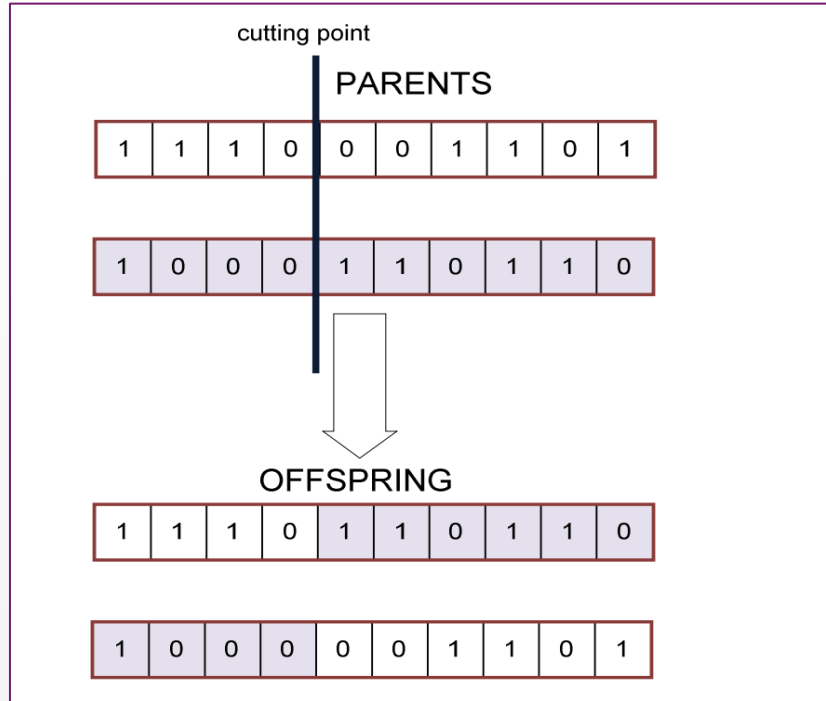
During the recombination of binary variables only parts of the individuals are exchanged between the individuals. Depending on the number of parts, the individuals are divided before the exchange of variables (the number of cross points).

1-Single point crossover

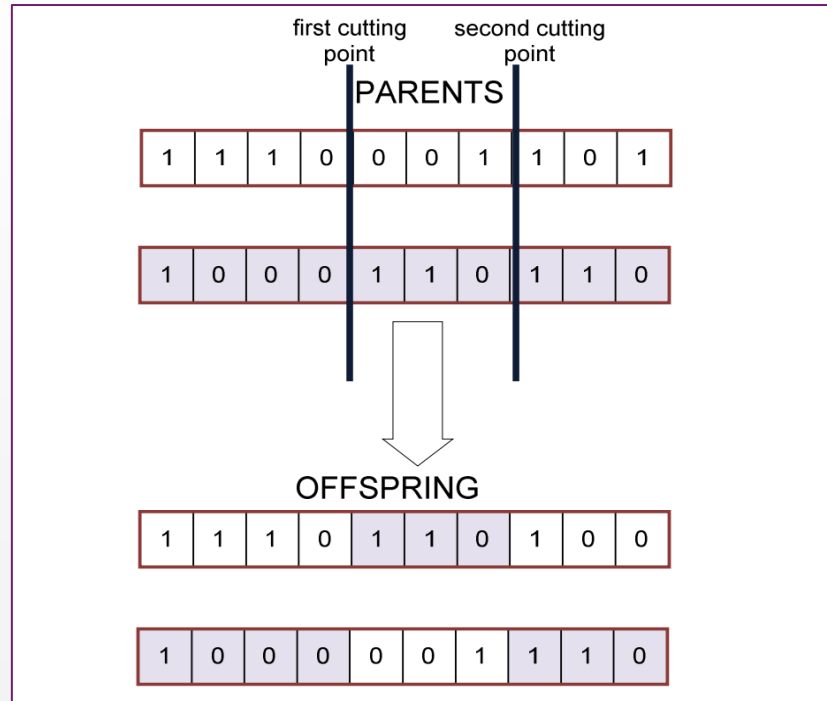
2-Double point crossover

3-Uniform crossover

1-Single point crossover



2-Double point crossover



3-Uniform crossover

Uniform crossover works by treating each gene independently and making a random choice as to which parent it should be inherited. This is implemented by generating a string of random variables from a uniform distribution over $[0, 1]$ whose size is equal to the size of an individual from the population. In each position, if the value is below a parameter p (usually 0.5), the gene is inherited from the first parent; otherwise from the second. The second off

If we have the parents of size 10:

parent 1	1	1	1	1	1	1	1	1	1	1	1
parent 2	0	0	0	0	0	0	0	0	0	0	0

and the string of random variables:

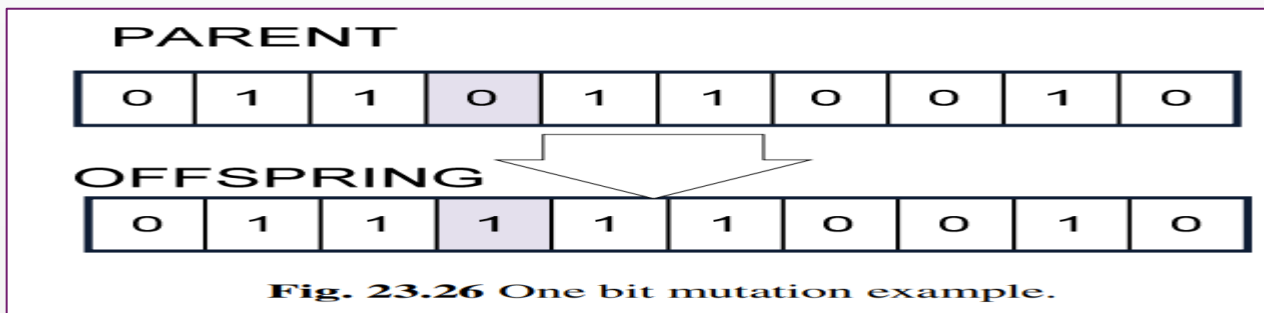
[0.17, 0.80, 0.33, 0.45, 0.51, 0.97, 0.12, 0.66, 0.73, 0.23]

then the offspring obtained after crossover are:

offspring 1	1	0	1	1	0	0	1	0	0	1
offspring 2	0	1	0	0	1	1	0	1	1	0

Mutation for Binary Representation

- By mutation individuals are randomly altered. These variations (mutation steps) are mostly small. Mutation is only applied to one individual and will produce one offspring. They will be applied to the variables of the individuals with a low probability (mutation probability or mutation rate). Normally, offspring are mutated after being created by recombination
- For binary valued individuals' mutation means the flipping of variable values, because every variable has only two states. Thus, the size of the mutation step is always 1.



Problem Scenario

You're planning a backpacking trip and have a knapsack with a maximum weight capacity of 15 kg. You need to select items to maximize the total value (usefulness) of what you bring.

Available Items

Item	Weight (kg)	Value (1-10)
Tent	4	9
Sleeping Bag	3	8
Food	5	7
Stove	2	5
Water Filter	1	6
First Aid Kit	2	4
Camping Chair	3	3
Extra Clothes	2	4

[1, 0, 1, 1, 0, 1, 0, 0] means:

- Tent: Yes (1)
- Sleeping Bag: No (0)
- Food: Yes (1)
- Stove: Yes (1)
- Water Filter: No (0)
- First Aid Kit: Yes (1)
- Camping Chair: No (0)
- Extra Clothes: No (0)

Total weight = 4 + 5 + 2 + 2 = 13 kg (within limit ✓)

Total value = 9 + 7 + 5 + 4 = 25

Genetic Algorithm Process

1. Initial Population

Randomly generate binary strings:

- Chromosome 1: [1,0,1,1,0,1,0,0] (weight: 13, value: 25)
- Chromosome 2: [1,1,0,1,1,0,0,1] (weight: 14, value: 32)
- Chromosome 3: [1,1,1,0,0,0,1,1] (weight: 17, value: 31) - **INVALID** (overweight)
- Chromosome 4: [0,1,1,1,1,0,1,0] (weight: 15, value: 29)

3. Selection

Select parents based on fitness (higher value = better chance):

- Parent A: [1,1,0,1,1,0,0,1] (value: 32)
- Parent B: [0,1,1,1,1,0,1,0] (value: 29)

4. Crossover (at point 4)

Parent A: [1,1,0,1 | 1,0,0,1]

Parent B: [0,1,1,1 | 1,0,1,0]

Child: [1,1,0,1 | 1,0,1,0]

5. Mutation (flip random bits)

Child before: [1,1,0,1,1,0,1,0]

Child after: [1,1,0,1,1,1,1,0] (bit 6 flipped
from 0→1)

Evolution Over Generations

- **Generation 1:** Best value = 32 ([1,1,0,1,1,0,0,1])
- **Generation 5:** Best value = 35 ([1,1,0,1,1,1,0,0]) - Tent, Sleeping Bag, Stove, Water Filter, First Aid Kit
- **Generation 10:** Optimal = 38 ([1,1,1,0,1,1,0,0]) - Tent, Sleeping Bag, Food, Water Filter, First Aid Kit (total weight: 15 kg exactly!)

Problem Scenario

Problem: Find x that maximizes $f(x) = x^2$, where $0 \leq x \leq 31$

Since x^2 is a simple quadratic function, we know the maximum is at $x = 31$ ($f(31) = 961$). Let's see how a Genetic Algorithm finds this!

Step 1: Problem Encoding

Binary Representation

Use 5 bits ($2^5 = 32$ possible values)

Binary $\rightarrow$ Integer mapping:

00000 = 0

00001 = 1

00010 = 2

...

11111 = 31

Individual: [1,0,1,1,0] = 22 decimal

$f(22) = 484$

Step 2: Create Initial Population (Random)

Individual 0: $[1, 0, 1, 1, 0] = 22 \rightarrow f(22) = 484$
Individual 1: $[0, 1, 0, 1, 1] = 11 \rightarrow f(11) = 121$
Individual 2: $[1, 1, 1, 0, 1] = 29 \rightarrow f(29) = 841$
Individual 3: $[0, 0, 1, 1, 0] = 6 \rightarrow f(6) = 36$
Individual 4: $[1, 0, 0, 1, 1] = 19 \rightarrow f(19) = 361$
Individual 5: $[0, 1, 1, 0, 1] = 13 \rightarrow f(13) = 169$
Individual 6: $[1, 1, 0, 1, 0] = 26 \rightarrow f(26) = 676$
Individual 7: $[0, 0, 0, 1, 1] = 3 \rightarrow f(3) = 9$
Individual 8: $[1, 0, 1, 0, 1] = 21 \rightarrow f(21) = 441$
Individual 9: $[0, 1, 1, 1, 0] = 14 \rightarrow f(14) = 196$

Step 3: Selection (Tournament Size = 3)

Randomly pick 3 individuals, select the best:

Tournament 1: Pick Ind2 (841), Ind5 (169), Ind9 (196) → Winner: Ind2 (841)

Tournament 2: Pick Ind0 (484), Ind6 (676), Ind3 (36) → Winner: Ind6 (676)

Tournament 3: Pick Ind4 (361), Ind1 (121), Ind8 (441) → Winner: Ind8 (441)

Step 4: Take two winners and perform crossover:

Parents:

Parent A (Ind2): [1,1,1,0,1] = 29 (f=841)

Parent B (Ind6): [1,1,0,1,0] = 26 (f=676)

Crossover at point 3:

Parent A: [1,1,1 | 0,1]

Parent B: [1,1,0 | 1,0]

Child 1: [1,1,1 | 1,0] = [1,1,1,1,0] = 30 → f(30) = 900

Child 2: [1,1,0 | 0,1] = [1,1,0,0,1] = 25 → f(25) = 625

Key Insights

1. Convergence Speed: Simple problem converges in 1-2 generations

2. Elitism: Preserves best solution found

3. Mutation: Provides final push from 30→31

4. Population Diversity: Maintained through crossover

Problem Scenario

A small factory can produce 5 different products. Each product requires specific machine time and generates profit. The factory operates 40 hours per week, and you need to select which products to produce to maximize weekly profit.

Product	Machine Hours per Unit	Profit per Unit (\$)	Market Demand (max units)
A	2	100	10
B	3	150	8
C	1	75	15
D	4	200	5
E	2.5	120	12

we need to represent **how many units** of each product

Encoding Scheme

Each product's quantity is represented by a binary number using enough bits to cover its maximum demand:

- Product A (max 10): needs 4 bits (0-15)
- Product B (max 8): needs 4 bits (0-15)
- Product C (max 15): needs 4 bits (0-15)
- Product D (max 5): needs 3 bits (0-7)
- Product E (max 12): needs 4 bits (0-15)

Total chromosome length: $4 + 4 + 4 + 3 + 4 = 19$ bits

Example Chromosome

Bits: [1010, 0110, 1111, 101, 1001]

Decoded:

- A: $1010_2 = 10$ units
- B: $0110_2 = 6$ units
- C: $1111_2 = 15$ units
- D: $101_2 = 5$ units
- E: $1001_2 = 9$ units

$$\begin{aligned}\text{Total hours} &= (10 \times 2) + (6 \times 3) + (15 \times 1) + (5 \times 4) + (9 \times 2.5) \\ &= 20 + 18 + 15 + 20 + 22.5 = 95.5 \text{ hours (EXCEEDS 40!)}\end{aligned}$$

2. Initial Population

Randomly generate 19-bit strings and decode:

Chromosome 1: [0100,0011,1010,010,0110] → Units: 4,3,10,2,6

Hours: $8+9+10+8+15 = 50\text{h}$ (invalid)

Profit: $\$400+\$450+\$750+\$400+\$720 = \2720 - penalty = $\$1720$

Chromosome 2: [0011,0010,0101,001,0011] → Units: 3,2,5,1,3

Hours: $6+6+5+4+7.5 = 28.5\text{h}$ (valid)

Profit: $\$300+\$300+\$375+\$200+\$360 = \1535

Chromosome 3: [0101,0100,0111,011,0100] → Units: 5,4,7,3,4

Hours: $10+12+7+12+10 = 51\text{h}$ (invalid)

Profit: $\$500+\$600+\$525+\$600+\$480 = \2705 - penalty = $\$1605$

Chromosome 4: [0010,0001,1000,010,1010] → Units: 2,1,8,2,10

Hours: $4+3+8+8+25 = 48\text{h}$ (invalid)

Profit: $\$200+\$150+\$600+\$400+\$1200 = \2550 - penalty = $\$1750$

جزاكم الله خيرا
وصل اللهم على سيدنا محمد وعلى آله وصحبه وسلم